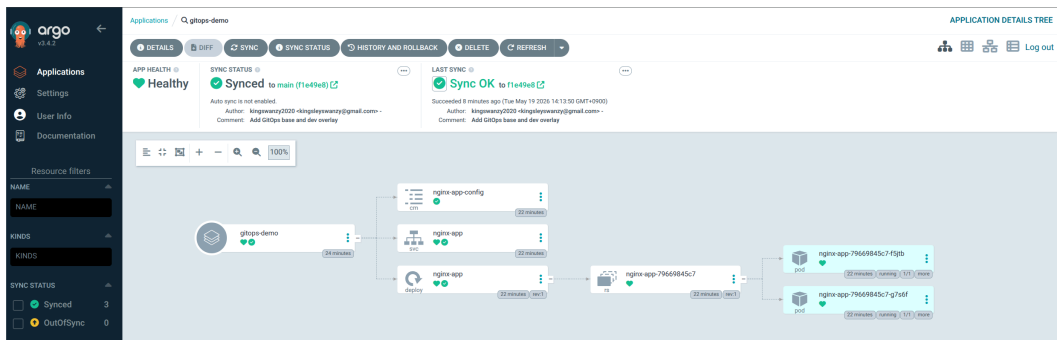


GITOPS DEPLOYMENT PIPELINE WITH ARGOCD

By Ahmed Tetteh | May 2026



Building a Production-Grade GitOps Pipeline

Project vision and goals

In this project, I'm building a production-grade GitOps pipeline where every cluster change flows through a pull request, so that I can have a complete audit trail and automatic drift correction.

Installing ArgoCD and Connecting to the Cluster

Setting up the continuous delivery engine

In this step, I'm setting up an ArgoCD CLI locally and deploy ArgoCD within my k8s cluster, so that I can execute infrastructure changes within the cluster.

```
king@king-TFG5577XG:~$ k get pods -n argocd
```

NAME	READY	STATUS	RESTARTS	AGE
argocd-application-controller-0	0/1	ContainerCreating	0	20s
argocd-applicationset-controller-b4bf95565-4n6ds	0/1	ContainerCreating	0	21s
argocd-dex-server-66cc8c85c6-x5w9d	0/1	Init:0/1	0	21s
argocd-notifications-controller-54d9b99856-zv7c5	0/1	ContainerCreating	0	21s
argocd-redis-7fbf6cdd5d-hmhwv	0/1	Init:0/1	0	20s
argocd-repo-server-77988ddcd9-psgvf	0/1	Init:0/1	0	20s
argocd-server-794b9c89f9-vn8r9	0/1	ContainerCreating	0	20s

Handling self-signed certificates

The `--insecure` flag is needed because the ArgoCD's default install uses a self-signed TLS certificate. Without it, the CLI rejects the connection.

Deploying the Sealed Secrets Controller

Enabling encrypted secrets in Git

In this step, I'm installing the Sealed Secrets Controller within my cluster, so that I can encrypt credentials before committing to Git.

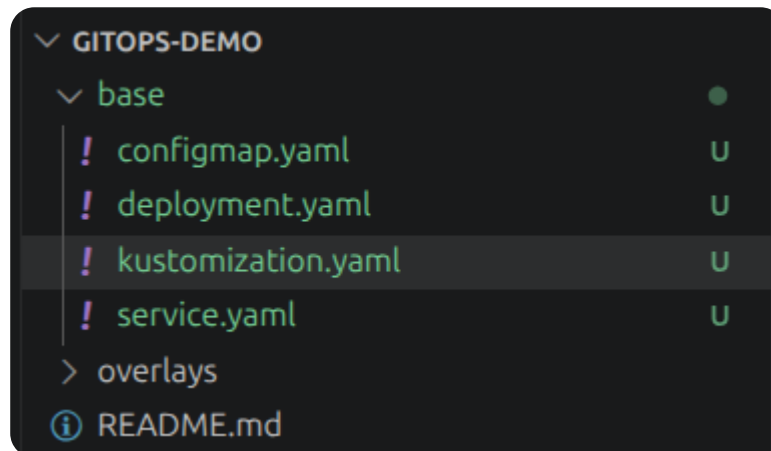
Verifying the controller deployment

I deployed the controller to the `kube-system` namespace, and confirmed it works by running checking in the `kube-system` namespace any pods with the label `"name=sealed-secrets-controller"`.

Structuring the GitOps Repository with Kustomize

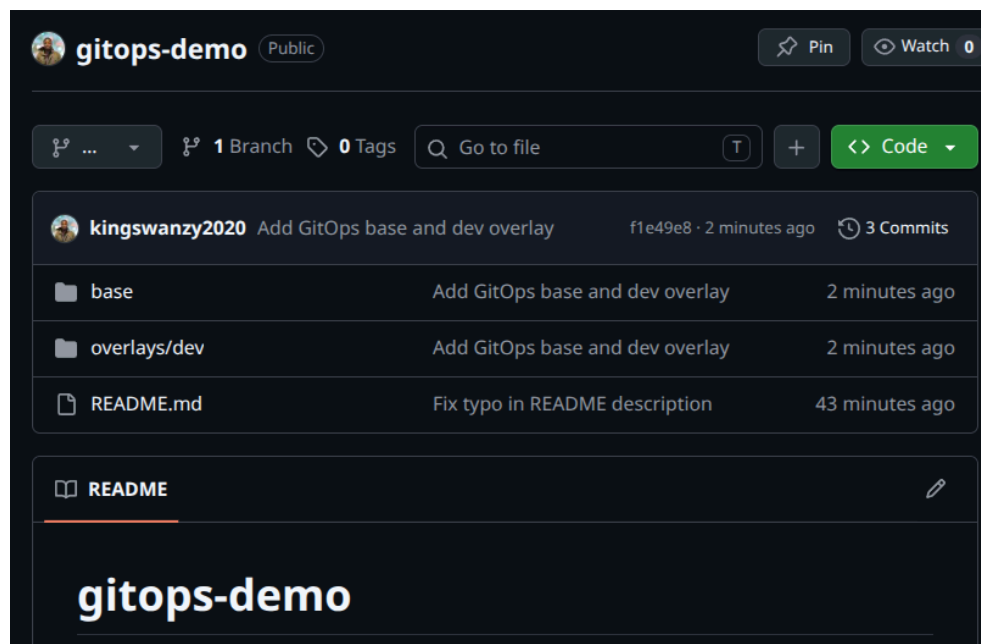
Designing the base and overlay layout

In this step, I'm setting up a GitOps Repository structure for my GitOps workflow, so that ArgoCD can refer to it as the single source of truth for the desired state of my cluster.



Understanding kustomization.yaml

The kustomization.yaml file acts as an entry point for Kustomize. It lists all the resource files that belong to this base. When ArgoCD processes this directory, it reads this file to know which manifests to include.



Syncing the Application with ArgoCD

Registering the repository and creating the Application

In this step, I'm connecting ArgoCD to my GitHub repository, so that ArgoCD can manage deployments to the cluster from Git.

Resources deployed on first sync

ArgoCD deployed two nginx pods to the dev namespace with kustomize overlay.

Enabling Auto-Sync and Demonstrating Self-Healing

Configuring automated reconciliation

In this step, I'm enabling Auto-sync and Self-Healing so that the cluster can revert back to its previous state if a manual change is done without referring to Git.

```
Name: argocd/gitops-demo
Project: default
Server: https://kubernetes.default.svc
Namespace: dev
URL: https://localhost:8080/applications/gitops-demo
Source:
- Repo: https://github.com/kingswanzy2020/gitops-demo
  Target: main
  Path: overlays/dev
SyncWindow: Sync Allowed
Sync Policy: Automated (Prune)
Sync Status: Synced to main (f1e49e8)
Health Status: Healthy
```

GROUP	KIND	NAMESPACE	NAME	STATUS	HEALTH	HOOK	MESSAGE
apps	Deployment	dev	nginx-app	Synced	Healthy		deployment.apps/nginx-app configured
	ConfigMap	dev	nginx-app-config	Synced			
	Service	dev	nginx-app	Synced	Healthy		

Drift detection in action

When I scaled the deployment, ArgoCD detected manual change and immediately terminated the pod, reversing the deployment back to the desired state as known from Git.

Encrypting Secrets Safely with Sealed Secrets

Encrypting database credentials for Git

In this step, I'm encrypting a kubernetes secret with kubeseal, so that I can safely commit it to GitHub.

```
king@king-TFG5577XG:~$ kubectl get secret db-credentials -n dev
```

NAME	TYPE	DATA	AGE
db-credentials	Opaque	1	3m59s

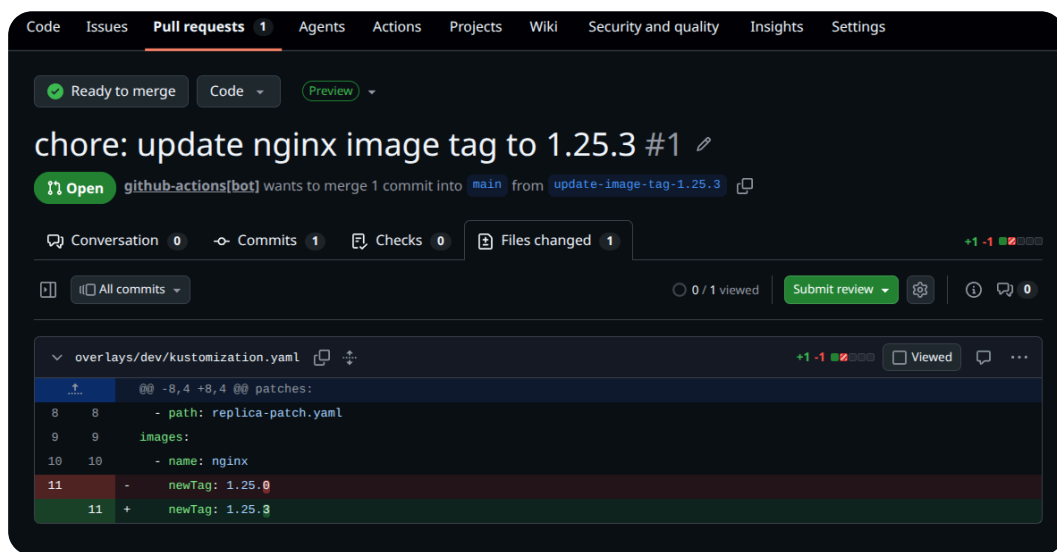
Why SealedSecrets are safe to commit

A SealedSecret is safe because it can only be decrypted by the Sealed Secret Controller with its private key within your cluster, while a regular Secret is not safe to commit to Git, and can be easily decrypted by others with access to the repository.

Automating Image Updates via GitHub Actions PRs

Creating the image tag update workflow

In this step, I'm creating a GitHub Actions workflow so that image updates flow through a Pull Request, then merged to the main branch for ArgoCD to perform the new changes within my cluster.



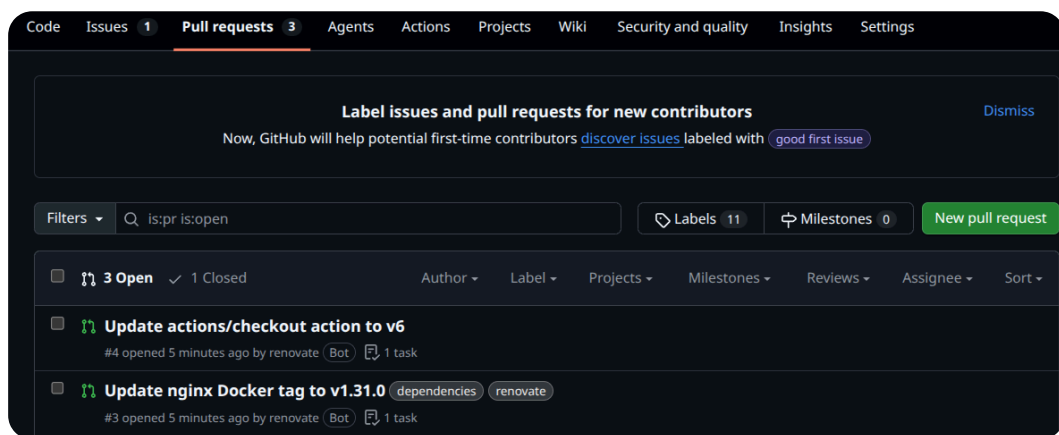
PR-based audit trail vs direct kubectl

Opening a PR improves the audit trail because it comes with a ability to view the diff, author, timestamp and approval history, so that a break within the cluster can be traced using that commit and reversed.

Configuring Renovate Bot for Dependency Automation

Automating dependency update pull requests

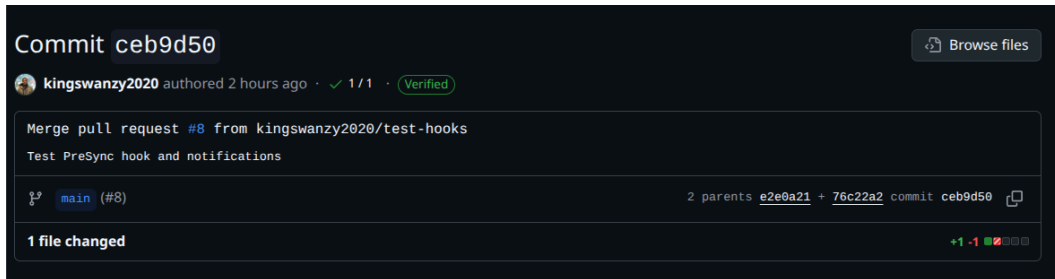
In this step, I'm configuring Renovate Bot for dependency updates by automatically opening a PR upon the release of updates for a base image or other dependencies.



Configuring the Kubernetes manager file patterns

The kubernetes manager needs fileMatch because without fileMatch, Renovate will skip your manifests because the kubernetes manager has no default file patterns.

Secret Mission: PreSync Hooks and Deployment Notifications



Improving deployment safety with PreSync hooks

In this project extension, I learned that PreSync hooks lets you run a job or hook prior to the main sync, so that if the hook fails, the entire sync is aborted.

Reflections and Key Takeaways

Tools and concepts mastered

The key tools I used include ArgoCD, Sealed Secrets, GitHub Actions, Renovate Bot, Helm and Git/GitHub. Key concepts I learnt include how to:

1. Install ArgoCD as your cluster's continuous delivery engine with auto-sync and self-heal, ensuring your cluster always matches Git and reverts any manual drift.
2. Encrypt credentials with Sealed Secrets and committed them safely to your GitOps repository, structured with Kustomize base and environment overlays.
3. Automate deployments through PR-based workflows using GitHub Actions for image tag updates and Renovate Bot for dependency updates.

Time and challenges

This project took me approximately 7 hours. The most challenging part was establishing the end-to-end automated workflow with accurate audit trails infrastructure change review through pull requests.

Looking ahead

I did this project today to learn how to how ArgoCD is used in combination with GitOps to design resilient and fault-tolerant architectures. Another skill I want to learn is how to deploy a multi-tier app production level app on Amazon EKS.